SOFTWARE REQUIREMENTS SPECIFICATION (SRS)
FOR

Team Name: Bit Crew

Class: COMP 490

Instructor: Edmund Dantes

Revision History

Revision Letter	By	Change Description	Date
1/A	Bit Crew	Initial draft of the SRS	11/9/25
B	Bit Crew	Final draft of the SRS	4/12/26

Table of Contents

Table of Figures

Page

NO TABLE OF FIGURES ENTRIES FOUND.

List of Tables

Page

1. INTRODUCTION

1.1 Scope

- Key features
    - Task management - create, edit, and delete tasks
    - Calendar integration - calendar where you can add events and see what works with your schedule
    - Reminders & notifications - push notifications for upcoming tasks
    - AI helper - feature that assists with task planning

1.2 Product Value

This app:

- Provide personalized productivity
- Sends alerts (for Android users) to prevent missing important dates and times

1.3 Intended Audience

Students

1.4 Intended Use

This app helps students organize academic and personal responsibilities by combining task management, calendar scheduling, and reminder features on one UI.

2. FUNCTIONAL REQUIREMENTS

FUNC_SRS_001: Launch Screen

Upon opening the app, users will be greeted by a launch screen displaying the AI Schedule Planner logo and branding elements while the app initializes. The launch screen will transition automatically to the login screen after system loading is complete.

FUNC_SRS_002: User Authentication

The Login screen shall allow users to access their personal accounts by entering their registered email and password. Users can log in using email/password or username/password. Upon successful login, a JWT token will be issued and stored client-side to maintain the user session.

FUNC_SRS_003: User Registration

New users shall be able to create an account by providing a username, email address, and password. The system will securely hash the password before storing it.

FUNC_SRS_004: Calendar Integration

The application will include an interactive calendar feature that enables users to view, create, modify, and delete tasks and events. The calendar shall display daily, weekly, and monthly views.

FUNC_SRS_004.1 – The app shall deliver push notifications to remind users of upcoming tasks and deadlines. Notifications are delivered on Android via Expo Push Notification Service (internally using FCM). iOS push notifications are not in the current scope.

FUNC_SRS_005: AI Scheduling and Task Planning

The AI component shall engage the user in a guided, task-based conversation to collect the information necessary to create a scheduled task.

FUNC_SRS_005.1 – The AI shall collect five required task fields: task name, day, start time, duration hours, and duration minutes.

FUNC_SRS_005.2 – Once all required fields are collected, the system shall generate a task draft for user confirmation, then create the task via the backend API.

FUNC_SRS_006: Help and Support Section

The application shall include a “Help & Contact Us” section accessible from the main menu.

FUNC_SRS_006.1 – The Help section will contain frequently asked questions, tutorials, and troubleshooting guides.

FUNC_SRS_006.2 – The Contact Us feature will provide a form for users to submit support requests and feedback directly within the app.

FUNC_SRS_007: Task Priority

Each task shall have an assigned priority level (low, medium, or high). Priority shall influence the order in which the scheduling service place tasks within a time window.

FUNC_SRS_008: AI Conversation Session

The AI assistant shall maintain conversation state across multiple messages within a session, tracking which of the five required task fields have already been collected so the user is not asked for the same information twice.

FUNC_SRS_009: Calendar State Management

Task placement on the calendar, including start time, end time, and scheduled status, shall be managed as local client state via Zustand. This data is persisted to the device's local storage but is not synced to the backend.

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interface Requirements

EXTINTF_SRS_001.1: Mobile app design guidelines

The app shall adhere to its respective mobile app design guidelines for each supported platform:

- iOS: Human Interface Guidelines (HIG)
- Android: Material Design Guidelines

EXTINTF_SRS_001.2: Main screens

The app shall have 3 main screens as described below:

1. Launch Screen
    - Static splash screen signifying that the app is loading.
2. Log-in Screen
    - Sign-up and/or authenticate and give access to a user's account.
3. Home Screen
    - The app's main navigation screen where all the app's related settings, functions and features reside.

EXTINTF_SRS_001.3: Home screen

The app's Home Screen shall follow the layout described below with respect to their mobile app guidelines:

- Home screen has a calendar-like design.
- 3 frame sub-layout: top bar frame, body frame, and bottom bar frame. -

Top bar frame contains the following:

- Settings button for the app, calendar and user profile settings.
- Month and year display-text and change button

- Body frame contains the following:

    - Calendar view
- Bottom bar frame contains the following:
    - AI Assistant/Bot interactive button
    - Activity (task) management button
    - Search button

3.2 Hardware Interface Requirements

EXTINTF_SRS_002.1: Supported devices

The app shall be able to run on the iPhone (iOS), iPad (iPadOS) - landscape orientation supported, Android smartphones, and Android tablets - landscape orientation supported.

EXTINTF_SRS_002.2: Network requirements

The app requires an active internet connection via cellular or Wi-Fi for the AI assistant, push notifications, and task sync. Basic calendar viewing of previously loaded tasks may be available offline depending on local state cached by the app.

EXTINTF_SRS_002.3: Communication Protocols

The app shall use the following communication protocols:

- HTTPS for frontend-to-backend communication:
    - RESTful API
- FCM (Android push notifications), routed via Expo Push Notification Service
- Local HTTP for backend-to-Ollama communication (internal to VPS)
- JWT for session authentication
- TLS for encryption
- JSON for all message formats

3.3 Software Interface Requirements

EXTINTF_SRS_003.1: Frameworks

The following framework shall be used:

- Frontend framework: Expo + React Native
- Backend: FastAPI (Python) served by Uvicorn

EXTINTF_SRS_003.2: Libraries

The following libraries shall be used:

- useWindowDimensions API

- Zustand
- Expo Router
- React Navigation
- Axios
- SQLAlchemy
- python-jose
- passlib/bcrypt
- expo-notifications

EXTINTF_SRS_003.3: AI Server Dependency

The AI assistant feature requires the Ollama service (qwen2.5:1.5b) to be running on the backend server. If the Ollama service is unreachable, the app shall display an appropriate error message and allow the user to continue using non-AI features.

EXTINTF_SRS_003.4: Backend Database

The system shall use an SQLite database managed via SQLAlchemy ORM. The database file shall be automatically created on first server startup. It shall store users, activities(tasks), and schedules.

3.4 Communication Interface Requirements

EXTINTF_SRS_004.1: The app shall communicate with Expo Push Notification Service to deliver push notifications to Android users. The backend sends notification payloads to Expo's API, which routes them to FCM for delivery.

EXTINTF_SRS_004.2: The backend shall communicate with the locally hosted Ollama service via HTTP on the server's loopback interface. Requests include a structured JSON prompt; responses return extracted task field data.

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Security

Core security requirements we will be mindful of:

NONFUNC_SRS_001.1: Secure authentication - validating users to make sure they are who they say they are

NONFUNC_SRS_001.2: Data encryption - scrambling information so only authorized users can access it

NONFUNC_SRS_001.3: Code protection - prevent anyone on the user side from tampering with the app's functionality or data

4.2 Capacity

NONFUNC_SRS_002.1: The app uses a SQLite database hosted on a VPS. Storage requirements are minimal: each task record is a small JSON/row with text fields. The current VPS provides sufficient storage for the expected user base of the app.

4.3 Compatibility

NONFUNC_SRS_003.1: The app shall be compatible with:

- Android 13 (API level 33) and above
- iOS 17 and above
- Both portrait and landscape orientations

4.4 Reliability

NONFUNC_SRS_004.1: The app's availability is dependent on the uptime of the hosted backend server. The deployment has no redundancy or failover. MTBCF has not been formally measured.

4.5 Scalability

NONFUNC_SRS_005.1: The server deployment is designed to support a small number of concurrent users. Simultaneous AI requests may result in increased response times due to shared Ollama resources. Scaling to larger user loads would require upgraded server hardware or a multi-instance deployment, which is outside the current project scope.

4.6 Usability

NONFUNC_SRS_006.1: The app shall be operated via touchscreen on mobile and tablet devices. Landscape orientation shall be supported for tablets to accommodate wider screen layouts.

5. QUALIFICATION PROVISIONS

Table I. Requirements Verification

SRS Req. ID	Paragraph Title	Verification Method
FUNC_SRS_001	Launch screen	Inspection
FUNC_SRS_002	User authentication	Demonstration
FUNC_SRS_003	User registration	Demonstration
FUNC_SRS_004	Calendar integration	Demonstration
FUNC_SRS_004.1	The calendar will provide reminders and notifications for upcoming tasks and deadlines	Demonstration
FUNC_SRS_005	AI scheduling and task planning	Demonstration
FUNC_SRS_005.1	AI collects five required task fields	Demonstration
FUNC_SRS_005.2	Task draft generation and backend creation	Demonstration
FUNC_SRS_006	Help and Support Section	Inspection
FUNC_SRS_006.1	The Help section	Inspection
FUNC_SRS_006.2	The Contact Us feature	Inspection
FUNC_SRS_007	Task Priority	Demonstration
FUNC_SRS_008	AI Conversation session	Demonstration
FUNC_SR	Calendar state management	Demonstration

S_009		
EXTINTF_SRS_001.1	Mobile app design guidelines	Inspection
EXTINTF_SRS_001.2	Main screens	Inspection
EXTINTF_SRS_001.3	Home screen	Inspection

SRS Req. ID	Paragraph Title	Verification Method
EXTINTF_SRS_002.1	Supported devices	Demonstration
EXTINTF_SRS_002.2	Network requirements	Test
EXTINTF_SRS_002.3	Communication protocols	Inspection
EXTINTF_SRS_003.1	Frameworks	Inspection
EXTINTF_SRS_003.2	Libraries	Inspection
EXTINTF_SRS_003.3	AI server dependency	Test
EXTINTF_SRS_003.4	Backend database	Inspection
EXTINTF_SRS_004.1	Expo Push Notification Service	Demonstration
EXTINTF_SRS_004.2	Ollama communication interface	Inspection
NONFUNC	Secure Authentication – Validating users to make sure they are who they say they are.	Demonstration

_SRS_001. 1		
NONFUNC_SRS_001. 2	Data Encryption – Encrypting data so only authorized users can access it.	Demonstration
NONFUNC_SRS_001. 3	Code Protection – Prevent anyone on the user side from tampering with the app's functionality or data.	Demonstration
NONFUNC_SRS_002. 1	Capacity	Inspection
NONFUNC_SRS_003. 1	Compatibility- iOS 17+, Android 13+	Demonstration
NONFUNC_SRS_004. 1	Reliability	Inspection
NONFUNC_SRS_005. 1	Scalability	Inspection
NONFUNC_SRS_006. 1	The app can be interacted with through touch screen	Inspection

6. NOTES

6.1 Acronyms and Abbreviations

Table II. Acronyms and Abbreviations

Abbreviation	Full name
AI	Artificial intelligence
API	Application programming interface
App	Application
FCM	Firebase cloud messaging
HTTP	Hypertext transfer protocol
ID	Identification
iOS	iPhone operating system
JSON	JavaScript object notation
MTBCF	Mean time between critical failures
JWT	JSON Web Token
OS	Operating system
TLS	Transport layer security
Wi-Fi	Wireless Fidelity
LLM	Large Language Model
UI	User Interface
VPS	Virtual Private Server
ORM	Object-Relational Mapping